🧬 Edit History

(Editar o histórico de commits)

👉 Aqui você aprende a:

- corrigir commits
- juntar commits
- voltar no tempo
- recuperar coisas perdidas

🧠 Ideia central

O histórico do Git **não é fixo** — você pode modificar

⚠️ Mas com cuidado:

- isso pode quebrar o projeto se usado errado

💧 1. `git commit --amend` — corrigir último commit

📌 Comando:

```
git commit --amend
```

🧠 O que faz?

Permite:

- mudar mensagem
- adicionar arquivos esquecidos

🔧 Exemplo

Você fez:

```
git commit -m "login"
```

Mas esqueceu um arquivo:

```
git add auth.js
git commit --amend
```

👉 Ele atualiza o commit anterior

💡 Resultado

Em vez de criar outro commit, ele **corrige o último**

⚠️ Regra

Não use se já fez `git push`

🔄 2. `git reset HEAD^` — desfazer commit

📌 Comando:

```
git reset HEAD^
```

🧠 O que faz?

👉 Remove o último commit

👉 Mantém os arquivos modificados

🔧 Exemplo

```
git commit -m "erro"
git reset HEAD^
```

👉 commit desaparece, código continua

💡 Uso

- corrigir commit errado
- reorganizar commits

📌 3. `git reset --hard <commit>`

📌 Comando:

```
git reset --hard a1b2c3
```

🧠 O que faz?

👉 Volta o projeto exatamente para aquele commit

⚠️ Perigo

- apaga commits posteriores
- apaga alterações

💧 Uso real

👉 “quero voltar para versão estável”

🧬 4. `git rebase -i` — edição avançada (nível profissional)

📌 Comando:

```
git rebase -i HEAD~3
```

🧠 O que faz?

Abre um editor com commits:

```
pick a1 commit 1
pick b2 commit 2
pick c3 commit 3
```

🔧 Você pode:

- reordenar commits

- apagar commits
- juntar commits

🔥 5. Squash (juntar commits)

Dentro do rebase:

```
pick a1 commit 1
fixup b2 commit 2
fixup c3 commit 3
```

🧠 Resultado

👉 vários commits viram um só

💡 Uso profissional

Antes:

```
"teste"
"ajuste"
"corrige erro"
```

Depois:

```
"implementa sistema completo"
```

🔍 6. `git reflog` — recuperar o que parecia perdido

📌 Comando:

```
git reflog
```

🧠 O que faz?

👉 mostra tudo que aconteceu no Git

Mesmo:

- commits apagados
- resets
- rebases

🔧 Exemplo

```
git reflog
git reset --hard a1b2c3
```

💡 Uso

👉 “perdi tudo” → ainda dá pra recuperar

🧠 Visual mental

```
Histórico original:
A — B — C — D
```

```
Após edição:
A — B — D' (modificado)
```

⚠ Regras de ouro (IMPORTANTÍSSIMO)

❌ NÃO FAÇA:

- rebase em código já enviado
- reset em branch compartilhada

☑ FAÇA:

- editar histórico local
- limpar commits antes do push

💧 Exemplo completo (vida real)

```
git commit -m "teste"
git commit -m "ajuste"
git commit -m "corrige bug"
```

Organizando:

```
git rebase -i HEAD~3
```

Resultado:

```
"implementa funcionalidade completa"
```

🧠 Mentalidade profissional

Você usa isso para:

- manter histórico limpo
- organizar commits
- melhorar legibilidade
- preparar código para equipe

🚀 O que você aprendeu aqui

Você agora sabe:

- corrigir commits
- apagar commits
- reorganizar histórico
- recuperar dados

👉 Isso é nível avançado real